FitBuddy — Phase 3: Project Design

System architecture, modules, data flow, UI and database design

Purpose

• Prepared from the supplied FitBuddy project document. This document is a submission-ready phase artifact/template; it does not claim that implementation, testing, screenshots, or deployment have been completed unless you add evidence from your actual project.

High-Level Architecture

FitBuddy follows a modular architecture: Frontend (HTML + Jinja2) → FastAPI backend/routes → Gemini AI layer; FastAPI also communicates with SQLite through SQLAlchemy for persistence. The supplied document identifies the frontend templates as index.html, result.html, and all_users.html.

Module Design

• Frontend module: collects user information, displays generated plans, accepts feedback, and displays admin information.

• Routing module (routes.py): handles home, workout generation, feedback submission, and admin view routes.

• Workout AI module (gemini_generator.py): generates the structured 7-day workout plan.

• Nutrition AI module (gemini_flash_generator.py): generates a concise nutrition/recovery tip.

• Update module (updated_plan.py): sends the original plan plus feedback to Gemini for revision.

• Database module (database.py): stores user and plan information with SQLAlchemy and SQLite.

Route Design

• / — displays the homepage.

• /generate-workout — receives user input, calls workout and nutrition functions, stores data, and renders results.

• /submit-feedback — retrieves the original plan, sends it with feedback to the update function, stores the revision, and renders the updated result.

• /view-all-users — retrieves users/plans and renders the admin view.

Database Design

User/plan records should support the supplied fields: user ID, name, age, weight, goal, intensity, original plan, and updated plan where applicable. The exact SQLAlchemy model fields should match the implementation actually used in your code.

UI Design

• index.html: user input form.

• result.html: user details, generated plan, nutrition tip, and feedback form.

• all_users.html: administrative table showing user information and original/updated plans.

• The supplied design calls for Flexbox, mobile responsiveness, consistent buttons, rounded elements, readable result blocks, and a fitness-oriented visual style.

Data Flow

User → index.html → FastAPI form processing → Gemini generation → SQLite storage → result.html. For revision: User feedback → /submit-feedback → original plan + feedback → Gemini → updated database record → result.html.